Moving the Mesh Without Remaking It

Louis Moresi [1]

[1] Australian National University

Keywords Underworld Code, Tricks of the Trade, development

A geodynamic model with a uniform mesh, almost certainly expends most of its resolution in the wrong places. The features that need small cells: a thermal boundary layer; a shear band; a subducting slab's megathrust zone, each occupy a few percent of the domain and usually move around during the calculation. Mesh refinement is important, and so is the ability to respond to changes in where the refinement is needed.

When the requirements for refinement change throughout the computation, we have to undertake an *adaptive mesh refinement* each step, or every few steps. Conceptually, the simplest way to do this is to build a new mesh with exactly the resolution we need, then transfer all the problem data to the new mesh. This will give us the ideal mesh for the problem, but it does mean navigating and interpolating between meshes and is particularly tricky in parallel.

An alternative is to *add* resolution to the mesh where and when it is needed, starting from a base mesh which we keep as a reference point for the duration of the calculation. Much of the mesh then remains unchanged and we only need to move data in areas where nodes have been added. Because we are limited to splitting existing elements, however, there are some limits to how "nice" a mesh we can construct.

An alternative to these form of refinement is to deform the grid so the nodes **bunch up** where additional resolution is required and become more widely spaced elsewhere. A fixed node budget, deployed as effectively as possible while maintaining mesh quality.

We'll show how mesh-redisribution can be done, what sort of resolution improvements we can achieve, and discuss the advantages / disadvantages of this way of doing things. But first, some background.

1. What refinement charges

Refinement changes the points in the mesh, and in a parallel run three things follow.

The partition may no longer be balanced. New cells appear where the physics is interesting, which is to say unevenly, so the ranks that own the interesting region end up owning most of the work. Fixing that means repartitioning and migrating — cells, degrees of freedom, every field, and any particles riding along. That migration is not a detail of the implementation. It is communication proportional to how much the mesh changed, and it happens every time the mesh changes.

PUBLISHED Aug 12, 2026
ARTICLE UWTN 2026-011
VERSION 1.0.0
LICENCE CC-BY-4.0
UNDERWORLD underworld3 0.0.0
SOURCE https://www.underworldcode.org/moving-the-mesh-without-remaking-it/

Fields have to be transferred between meshes that share no nodes. The old and new meshes are related by the refinement rule, but the values live at places that do not correspond, and interpolating between them costs accuracy every time. Do it every ten steps for a thousand steps and the transfer error is a term in your answer.

The multigrid hierarchy must be re-derived. Remeshing or refining the existing mesh means creating a new hierarchical relationship between nodes on the nesting of the grids.

2. Distort the mesh, don't re-mesh

There is an alternative to re-meshing that does not require repartitioning or recontructing the hierarchical nature of the grid. It may not require remapping the data under some circumstances. We stretch the grid, like an elastic net, allowing the points to move but to stay connected to their neighbours.

When the node budget is fixed, no node is created, none is destroyed, and no cell changes its neighbours. Every node stays on the rank that owned it. What moves is where the nodes **are** — they slide to where the resolution is wanted and away from where it is not.

Much of what makes refinement complex and expensive is then absent by construction:

- the partition is unchanged, because ownership is a property of the point set and the point set did not change;
- the connectivity is unchanged, so the multigrid hierarchy — which is built from the topology, not from the coordinates — is still a hierarchy;
- there is no mesh-to-mesh transfer, because there is only one mesh.

What you give up is equally clear, and we will come back to that: a fixed budget of nodes can only be redistributed, never increased.

3. Equidistribution is not mesh optimisation

As with any simple idea, the practical implementation is often full of traps. Redistributing a mesh is a global operation: nodes all need to move in synchrony and they cannot overtake each other or the mesh becomes tangled an unusable. These are constraints that ultimately limit how much we can improve mesh resolution.

The algorithm that we need is *equidistribution*: supply a metric $M(x)$ — a monitor function saying how much resolution is wanted, and in a tensor metric, in which direction — and ask for the map from a fixed computational mesh to the physical mesh under which every cell carries the same amount of M. That is a Monge–Ampère-style problem, and it is the same target the refiner is chasing; the difference is only that the refiner is allowed to add nodes and we are not.

Underworld3 solves it with the Huang–Kamenski moving mesh PDE (Huang & Kamenski, 2015), which generates the physical mesh as the image of a fixed computational mesh under the inverse coordinate map, minimising

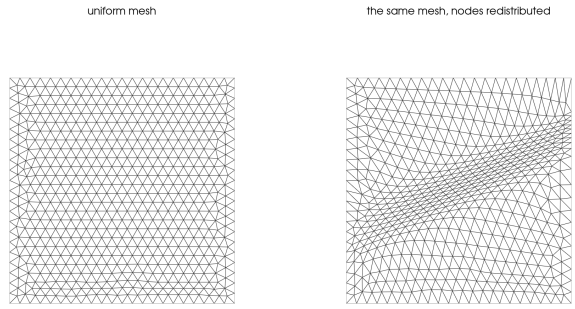
$$G = \theta\sqrt{\det M}\; S^q + (1-2\theta)\, d^{\,q}\, r^p (\det M)^{(1-p)/2}, \qquad q = \frac{d\,p}{2}, \tag{1}$$

with $S = \mathrm{tr}\big(\mathbb{J}\, M^{-1} \mathbb{J}^T\big)$, $\mathbb{J} = \hat{E} E^{-1}$ the Jacobian of the map between the reference and physical elements, and $r = \det \mathbb{J}$. The first term is the shape term, the second the size term.

Two properties of that functional are why this is the mover we use and not one of the several we retired.

It cannot fold the mesh. $G \to \infty$ as $\det \mathbb{J} \to 0$, so a cell cannot be driven through zero volume without the energy going to infinity first. That is a theorem about the functional (Huang & Kamenski, 2017), not a guard bolted on afterwards, and it is the difference between a mover you can leave running in a time loop and one you have to watch.

It aligns as well as clusters. M is a tensor, so a metric that is thin across a fault normal and long along it produces cells that are thin across the fault and long along it. An isotropic monitor function can only ask for smaller cells; a tensor one can ask for the *right* cells, which for a sheet-like feature is worth far more than the same node count spent isotropically.




```
  is uniform. On the right the same mesh has
its cells drawn in along a
  diagonal band, where they are noticeably
smaller and elongated along the
  band direction, and correspondingly larger
away from it.
```

(a) (b)

Figure 1: The same mesh, before and after redistribution to an anisotropic metric across a diagonal band. Both panels have **1152 cells and 621 nodes** — the counts are not merely similar, they are the same mesh object at two moments. What changed is where the nodes are: the median cell on the band is 1.65× smaller than in the bulk, and the cells there are visibly stretched along the band rather than simply shrunk. Nothing was refined, nothing was transferred between meshes, and no node changed rank.

In use it is two calls:

```python
import underworld3 as uw

# A scalar metric from |grad T|. refinement=R is the finest:coarsest
# cell-size ratio you are asking for, not a quality target.
rho = uw.meshing.metric_density_from_gradient(
    mesh, T, refinement=5, coarsening="auto",
    metric_choice="front-following")

# Move the nodes. The mover owns field transfer: it remaps T, carries
# the semi-Lagrangian history and fires the on_remesh hooks.
uw.meshing.node_redistribution(
    mesh, rho,
    method_kwargs=dict(step_frac=0.2, accel="none", momentum=0.0,
                       n_outer=400),
    slip_surfaces=True,      # boundary nodes slide tangentially
    skip_threshold=0.9)      # don't move an already-aligned mesh
```

`accel` selects how the outer iteration is driven. The unaccelerated descent above is the setting to reach for when the grading you get is weaker than the grading you asked for; it needs more outer iterations and it converges reliably. *Checking that the mover moved*, below, says how to tell a mover that has finished from one that has merely stalled.

4. Data updates are still needed

It would be nice to say no field transfer is needed but it's not true. Nodes move, so the value stored at a node is now the value of the old field at a place the node used to be, and it has to be re-evaluated. That is an interpolation and it costs us in accuracy once we evaluate the update.

What it does not cost is communication. A node moves a fraction of a cell diameter, so the point it must be evaluated at lies in its own patch or one owned by a neighbour — inside the halo the mesh already maintains. Compare the full remeshing case, where source and target are unrelated meshes and locating a point can land anywhere in the domain, on any rank, requiring a parallel search and then a migration to answer it. Same operation in name; entirely different in cost and in who has to talk to whom.

5. Setting a reference frame

The functional measures a cell's distortion against a *reference* element. By default, the reference is the mesh as it was on the first call. That is the correct choice for redistribution — we are asking for a map from this mesh to a better-graded one — and it has a consequence that is quite easy to miss.

Under a uniform metric, a distorted mesh is its own optimum. If $\hat{E} = E$ then $\mathbb{J} = I$ in every cell, the shape gradient is zero everywhere, and the mover does nothing. Not approximately, exactly. The measured displacement of `redistribute_nodes` under $M = I$ is exactly zero. Handed a mesh full of needle-shaped elements, for example, the redistributor does not drive towards a more evenly distributed mesh as, intuitively, we might at first expect.

But, if we replace each cell's reference element with a single *regular simplex*, scaled to that cell's own current volume, the same functional describes the distortion "away from equilateral" rather than "away from the mesh I was handed". Because each reference is scaled to the cell's own volume, $r = \det \mathbb{J} = 1$ at entry, so the size term starts at its optimum and stays there — the grading is preserved and only shape does work.

Three frames, then, from one piece of machinery:

`reference`	the reference element is	what it does
`"mesh"`	the mesh at first call	redistribute to a metric
`"ideal"`	a regular simplex at each cell's own volume	repair shape, hold size
`"ideal-metric"`	a regular simplex at one common volume	repair shape, metric sets size

The user-facing call is `mesh.relax()` for the metric-free case and `mesh.relax(M)` for the third, alongside `mesh.redistribute_nodes(M)` for the first.

> **`relax()` and `relax(metric)` are different operations**
>
> Metric-free relaxation is a shape guarantee: sizes are held, so shapes can only improve. Passing a metric re-grades the mesh, and re-grading will trade shape away to do it. On a four-level graded box the 99th-percentile maximum angle went 117.9° arrow.r 113.8° without a metric and 117.9° arrow.r **127.4°** with one. The metric version is not the better-informed version; it is a different job.

6. The ceiling is set by the node budget

A mover redistributes; it never adds a node. Starting from a uniform mesh it saturates — about 1.8× finer on a fault than in the background — and no amount of iteration goes past that. The nodes it would need are somewhere else, and if the metric has ridges between where the nodes are and where we want them, the algorithm cannot push nodes over that barrier.

We can see this limit directly if we add in the nodes that the redistribution needs when we initially mesh the domain. If we put a refinement into the base mesh, the mover does not merely maintain it, it **compounds** it. Measured on a fault in an annulus, as the ratio of on-fault to bulk nearest-neighbour spacing — lower is finer:

base mesh	base ratio	after the mover
uniform	1.00	0.55 (∼1.8×)
gmsh, `refine_lines` at 2	0.44	0.29 (∼3.4×)
gmsh, `refine_lines` at 3	0.30	0.19 (∼5×)

The MMPDE algorithm has the capacity to work with meshes that are already refined and improve them. It is helpful to have MMPDE even if you are also refining the mesh. There is usually benefit, and the mover is benign: it will not degrade or undo any refinement you give it. We will come back to this in a later article.

7. Where the two strategies meet

So the two are not really alternatives, and the second reference frame is where they meet. Once the base mesh carries refinement, or a subdivision has added some, the mover has a second job to do on it.

Refinement puts each new node where its tagging rule says, never where the geometry would want it, so a refined mesh carries needles and slivers that record the base mesh's arbitrary choices rather than anything about the problem. That is precisely what the ideal frame is for: keep the resolution the refinement installed, and move the nodes to where the shapes want them.

It is worth having. On an adapted mesh of 1143 cells, one `relax()` call cut the interpolation error of a localised feature by **17.5%** — no new cells, no new degrees of freedom, no change to the solver. On the production path, with Stokes preconditioned by a custom-prolongation full multigrid:

configuration	cells	error	velocity KSP its
adapt only	832	2.746e-2	4
relax at end	832	2.559e-2	3
relax every generation	856	**2.384e-2**	4

Relaxing once at the end costs nothing in cells and takes 6.8% off the error; relaxing inside the refinement loop, so that each pass marks from already-relaxed coordinates, takes off 13.2% for three percent more cells. How much either is worth depends on how poor the mesh was to begin with — on a base already near-optimal for its refinement rule there is little to repair, which is the honest reason these numbers are not larger.

8. Does the hierarchy actually survive?

That is the claim the whole approach rests on, so it should be checked rather than asserted. In the production comparison above, all three configurations kept a full four-level `pc=mg`, none fell back to algebraic multigrid,

all converged for the same reason, and their peak velocities agreed to four significant figures. Moving the nodes did not cost a level.

There is one real qualification. Multigrid does not care about coordinates — the hierarchy lives in the operators, and intermediate meshes are preserved as topology, not as geometry. But Underworld3's custom-prolongation transfers are *built* by geometric point location rather than derived from the refinement relation, and moving the nodes can leave a coarse degree of freedom with no fine image under the local-support barycentric builder. The build now retries with the global-support RBF builder before giving up. Measured in 3D: before the retry existed the hierarchy collapsed to algebraic multigrid at 23 iterations; with it, `pc=mg` at 2.

The topology survives, in other words, but a transfer operator built from geometry has to be told the geometry moved.

9. What it does not fix

Three limits, stated plainly.

Slivers that are structural. Refinement by centroid — the Alfeld strategy — places an interior point against a fixed parent face. The resulting sliver is a property of the *topology*, and no amount of node motion repairs a topology. Relaxed or not, cells with $q < 0.3$ stay at 25–28% and maximum angles at 176–178°. The mover is a shallow tool on that mesh; refine by bisection instead.

Accuracy in three dimensions. In 3D, relaxation improves mesh quality substantially — the near-degenerate population halves (cells with $q < 0.1$, 3.6% arrow.r 1.8%), median quality goes 0.32 arrow.r 0.39, the 99th-percentile dihedral angle comes back from 153° to 146° — and leaves interpolation error alone (+0.5%). That is consistent rather than disappointing: relaxation holds the size distribution, and in 2D the accuracy gain came from cells *aligning* onto the feature, which an isotropic metric in 3D gives them no reason to do. Use it in 3D for conditioning and element quality. An anisotropic 3D metric is the open question.

A single number for "wasted refinement". How much resolution ends up outside the region that asked for it is a natural thing to want as a scalar, and it resists being one: the property is spatial, since large coherent blocks of over-refinement matter and scattered cells do not, and every scalar we tried averaged that structure away. Look at the per-cell map of $\log_2(h/h_{\text{asked}})$ instead.

10. Checking that the mover moved

One practical point, because it is the failure mode this method has and it is not self-announcing.

A mover that stops early returns a perfectly valid mesh. It is non-folded, its topology is intact, its partition is unchanged, and every field on it is consistent — it is simply not the mesh you asked for. Nothing downstream can tell, and the symptom, *the grading looks weaker than the metric I supplied*, reads as a modelling problem rather than a solver one.

So check two things after a move, and check them the first time you set a problem up rather than only when something looks wrong.

The iteration trace. The mover reports its outer iterations under `verbose=True`. What you want to see is the functional decreasing and the line-search scale staying off zero. An outer step with `scale=0.000` and `dI=+0.00e+00` is not convergence in any useful sense: it means the line search rejected the trial direction and

backtracked to no step at all. If that appears after a handful of iterations out of a budget of a hundred and fifty, the mover stopped, it did not finish.

The grading you actually got. Take the median cell size inside the region the metric asked to refine, divide by the median well outside it, and compare against the ratio you requested. This is three lines of NumPy and it is the only direct evidence that the metric was honoured:

```
h = np.sqrt(cell_areas(mesh))            # cell size, per cell
on, off = in_feature(centroids), far_from_feature(centroids)
print("achieved %.2fx" % (np.median(h[off]) / np.median(h[on])))
```

Expect the answer to fall short of the metric, and by a lot from a uniform base — that is the fixed-budget ceiling of the previous section, and it is honest behaviour. What it should not do is come out near 1.0 while the mover reports success.

11. Using it

```
# Redistribute to a metric: grading, in the mesh's own reference frame.
mesh.redistribute_nodes(metric)

# Repair element shapes at fixed size: the ideal frame.
child = mesh.adapt(metric, max_levels=3)
child.relax()

# Or ask adapt to do it for you.
child = mesh.adapt(metric, max_levels=3, relax=True)             # at the end
child = mesh.adapt(metric, max_levels=3, relax="per-generation") # in the loop
```

All of these keep the vertex count, the degree-of-freedom layout and the parallel partition exactly as they were. That is the point of the whole exercise: the resolution follows the physics, and everything built on top of the mesh — the multigrid hierarchy above all — does not have to be told.

References

Huang, W., & Kamenski, L. (2015). A geometric discretization and a simple implementation for variational mesh generation and adaptation. *Journal of Computational Physics*, *301*, 322–337. https://doi.org/10.1016/j.jcp.2015.08.032

Huang, W., & Kamenski, L. (2017). On the mesh nonsingularity of the moving mesh PDE method. *Mathematics of Computation*, *87*(312), 1887–1911. https://doi.org/10.1090/mcom/3271